

ЛЕКЦИЯ 12

Эволюционные алгоритмы и адаптивные системы

Отбор, мутация и обучение как способ проектировать поведение роботов

две части по 45

ение распределёнными робототехническими системами»

План лекции

- | | | | |
|---|---|----|---------------------------------------|
| 1 | Зачем эволюция инженеру | 7 | Метод роя частиц |
| 2 | Генетический алгоритм: устройство | 8 | Эволюционная робототехника |
| 3 | Операторы отбора, скрещивания и мутации | 9 | Нейроэволюция контроллеров |
| 4 | Эволюционные стратегии и CMA-ES | 10 | Проблема назначения вклада |
| 5 | Дифференциальная эволюция | 11 | Разрыв между симуляцией и реальностью |
| 6 | Многокритериальная оптимизация | | |

Синие — первая часть · янтарные — вторая часть

ЧАСТЬ 1

Эволюция как метод оптимизации

Как отбор и случайность решают задачи, которые не берутся градиентом

“

Эволюция не ищет решение — она **выращивает** его. Там, где нет производной и нет модели, это иногда единственный доступный путь.

Ключевая мысль первой части

1

Когда классические методы не работают

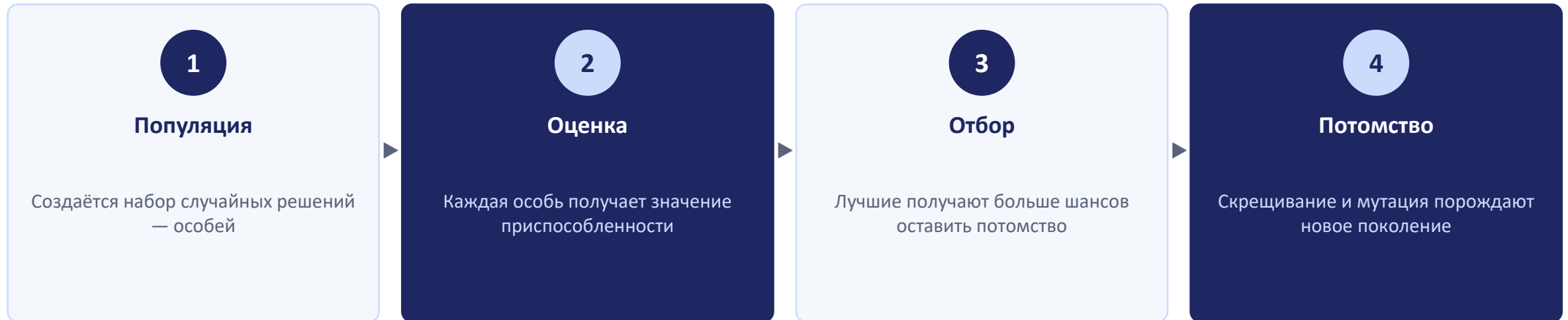
- Целевая функция не задана формулой, а измеряется экспериментом
- Производная не существует или её невозможно вычислить
- Пространство поиска содержит множество локальных минимумов
- Часть переменных дискретна, часть непрерывна

Пример

Качество поведения робота нельзя продифференцировать по весам нейросети напрямую: чтобы узнать значение целевой функции, нужно запустить робота и посмотреть, что получилось.

2

Цикл генетического алгоритма



Цикл повторяется, пока приспособленность не перестанет расти или не кончится бюджет.

- Двоичное: классика, удобна для комбинаторных задач
- Вещественное: естественно для параметров регуляторов
- Перестановочное: для маршрутов и расписаний
- Древовидное: для генетического программирования

Ключевая идея

Выбор кодирования важнее выбора операторов. Неудачное кодирование делает задачу нерешаемой для любого набора настроек.

Метод	Как работает	Свойство
Рулетка	Вероятность пропорциональна приспособленности	Чувствителен к масштабу функции
Турнир	Случайные k особей, побеждает лучшая	Устойчив, легко настраивается
Ранговый	Вероятность зависит от места в рейтинге	Не зависит от масштаба
Элитизм	Лучшие переходят в новое поколение без изменений	Гарантирует неухудшение

На практике чаще всего применяют турнирный отбор с элитизмом: он прост, устойчив и почти не требует настройки.

- Скращивание комбинирует признаки родителей — использует накопленное
- Мутация вносит случайное изменение — исследует новое
- Баланс между ними определяет судьбу поиска
- Слишком много мутаций — случайный поиск, слишком мало — застой

Ключевая идея

Это классическое противопоставление исследования и использования. Оно встретится нам ещё раз в лекции про обучение с подкреплением — там же под другими именами.



Как думаете?

Генетический алгоритм быстро вышел на плато и перестал улучшаться. Что стоит попробовать первым?

A

Увеличить размер популяции вдвое

B

Повысить вероятность мутации и проверить разнообразие популяции

C

Увеличить число поколений втрое

D

Сменить целевую функцию

? Как думаете?

Генетический алгоритм быстро вышел на плато и перестал улучшаться. Что стоит попробовать первым?

- A** Увеличить размер популяции вдвое
- B** Повысить вероятность мутации и проверить разнообразие популяции
- C** Увеличить число поколений втрое
- D** Сменить целевую функцию

ОТВЕТ

Проверить разнообразие: плато почти всегда означает преждевременную сходимость, когда популяция выродилась в клонов. Больше поколений здесь не помогут.

- Работают с вещественными векторами, мутация — гауссов шум
- Схемы $(\mu + \lambda)$ и (μ, λ) различаются судьбой родителей
- Сила мутации адаптируется в ходе поиска
- CMA-ES подстраивает всю ковариационную матрицу распределения

Факт

CMA-ES фактически выучивает форму оврага в ландшафте целевой функции и начинает делать шаги вдоль него — поэтому он остаётся эталоном для непрерывных задач без градиента.

$$\sigma \leftarrow \sigma \cdot c, \text{ если доля успешных мутаций} < 1/5$$

- σ — сила мутации, то есть разброс случайных изменений
- Мало успехов — шаги слишком велики, σ уменьшают
- Много успехов — можно двигаться смелее, σ увеличивают
- Простейший пример самоадаптации: алгоритм настраивает сам себя

Историческое правило Реченберга, с которого началась адаптация параметров.

- Новое решение строится из разностей между случайными особями
- Разности сами задают подходящий масштаб шага
- Очень мало настраиваемых параметров
- Один из лучших методов для непрерывной оптимизации без градиента

Ключевая идея

Масштаб шага не задаётся вручную, а берётся из самой популяции: пока решения разбросаны — шаги крупные, при сходимости они уменьшаются автоматически.

- Критериев обычно несколько: скорость, точность, расход энергии
- Улучшая одно, почти всегда портим другое
- Решение — множество Парето: набор несравнимых компромиссов
- NSGA-II строит его за один запуск алгоритма

Пример

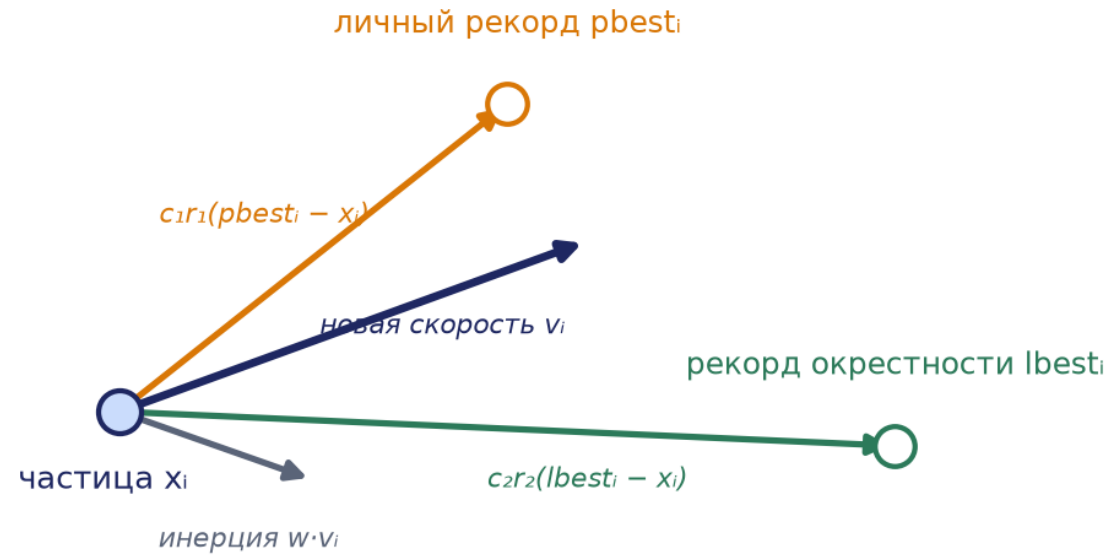
Быстрый робот тратит больше энергии. Единственного «лучшего» решения нет — есть фронт вариантов, из которого инженер выбирает по внешним соображениям.

- Частица помнит свой лучший результат и знает лучший у соседей
- Движение складывается из инерции, личного и социального притяжения
- Нет отбора и нет гибели особей — все выживают и учатся
- Работает быстрее генетического алгоритма на гладких задачах

Ключевая идея

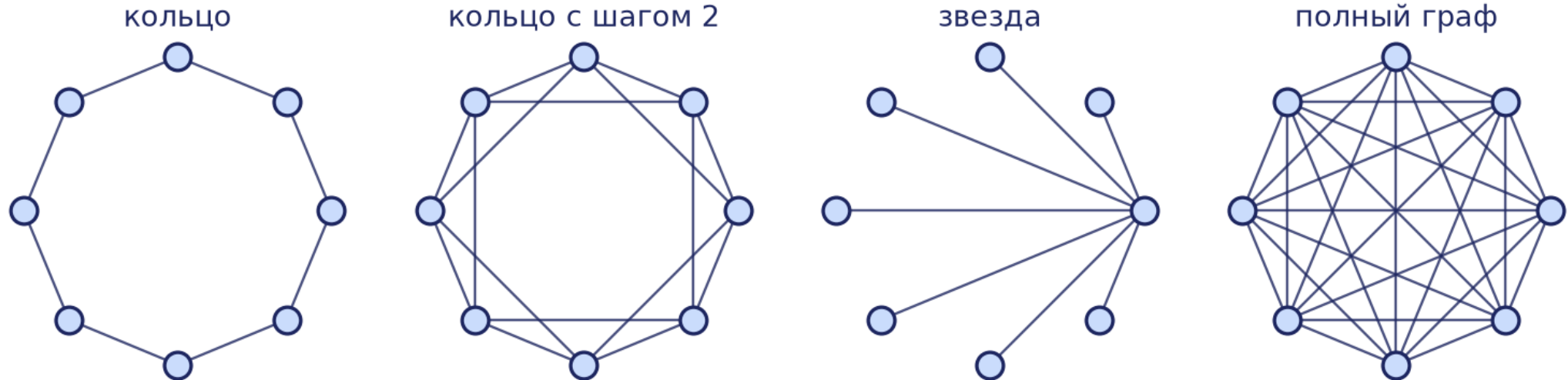
Генетический алгоритм отбирает лучших и уничтожает остальных. Рой частиц никого не уничтожает — он подтягивает всех к лучшему из найденного. Разная философия, схожие результаты.

Составляющие скорости частицы в методе роя частиц



Новое положение частицы складывается из трёх составляющих: инерции текущего движения, притяжения к собственному лучшему результату (p-best) и притяжения к лучшему результату соседей (n-best). Баланс коэффициентов определяет, будет рой исследовать пространство или уточнять найденное.

Топологии окрестностей роя: чем плотнее связь, тем быстрее сходимость и выше риск застрять



Географическое соседство определяется расстоянием в пространстве поиска, социальное — фиксированной структурой связей независимо от положения. Плотная топология ускоряет сходимость, но повышает риск застрять в локальном оптимуме.

П Е Р Е Р Ы В

Конец первой части

Мы разобрали эволюционные методы как инструмент оптимизации. После перерыва применим их к самому трудному объекту — поведению реального робота.

ЧАСТЬ 2

Эволюционная робототехника

Как вырастить контроллер робота и почему это труднее, чем кажется

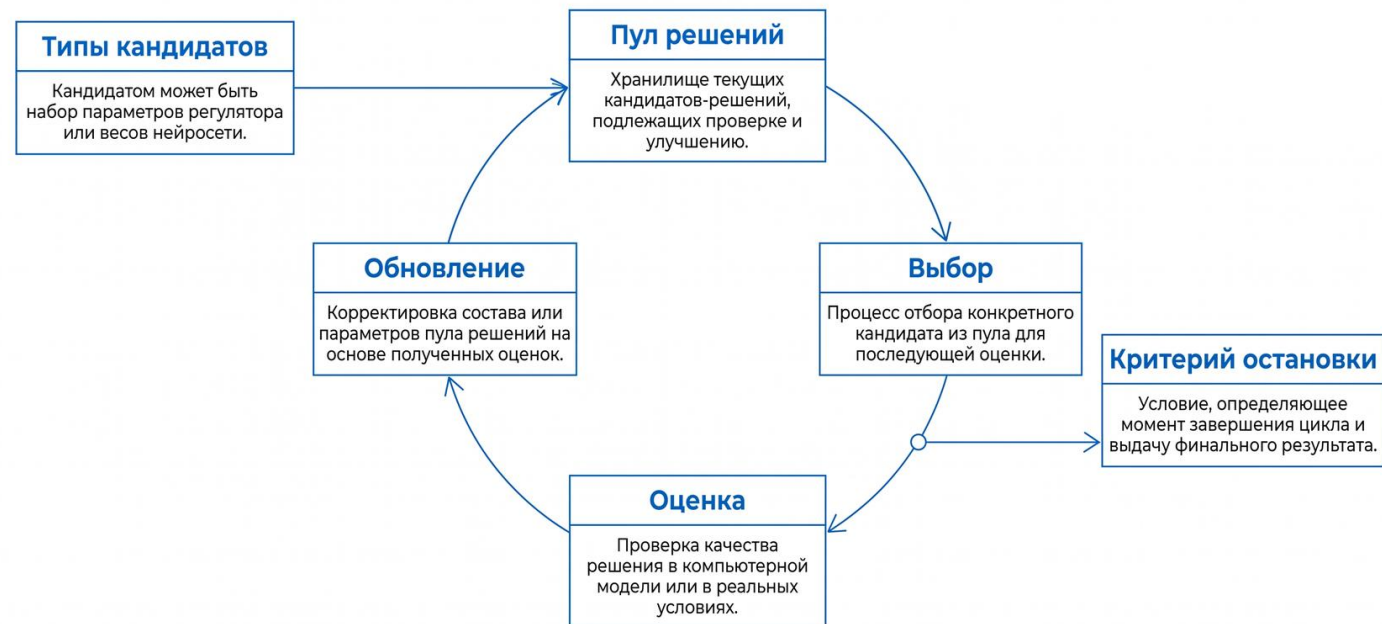
- Контроллер робота кодируется как набор параметров
- Приспособленность измеряется поведением в симуляции или на работе
- Отбор и мутация постепенно улучшают поведение
- Инженер задаёт цель, а не способ её достижения

Ключевая идея

Проектировщик перестаёт описывать, **как** робот должен действовать, и описывает только, **что** считается хорошим результатом. Способ находит алгоритм.

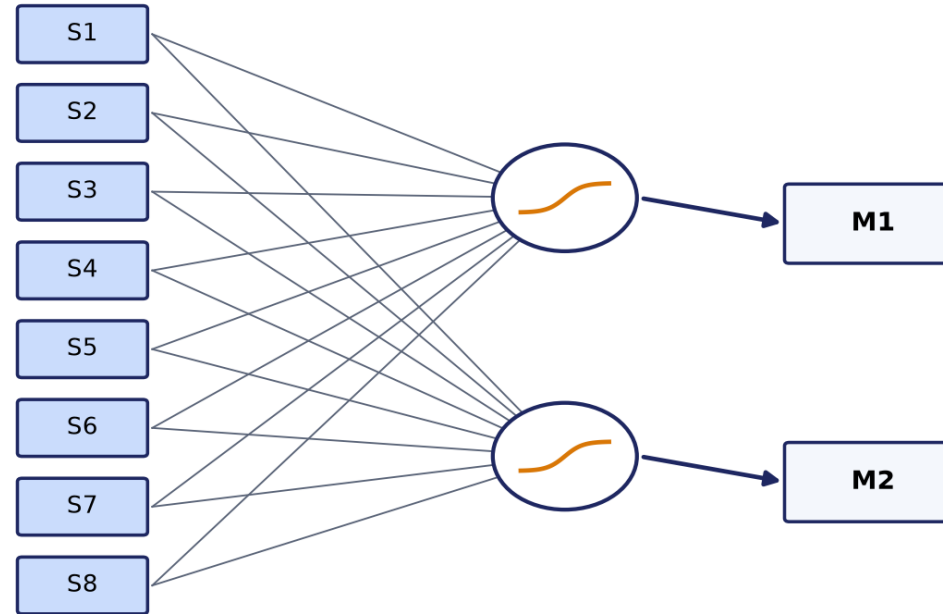
8 Цикл метаэвристической оптимизации контроллера

Цикл работы метаэвристического оптимизатора



Оптимизатор порождает набор кандидатов, каждый разворачивается в контроллер, испытывается в симуляции или на реальном роботе, получает оценку приспособленности, и цикл повторяется. Узкое место — именно испытание: оно занимает секунды и десятки секунд.

синаптические веса w_{ij} — то, что эволюционирует



S1...S8 — датчики
M1, M2 — моторы
левого и правого колеса

передаточная функция нейрона: $f(x) = 1 / (1 + e^{-x})$

Показания датчиков подаются на входы сети, выходы задают скорости колёс, а объектом оптимизации служат синаптические веса. Даже небольшая сеть на десятки весов способна породить содержательное поведение вроде обхода препятствий.

- Оптимизируются не только веса, но и топология сети
- NEAT наращивает сеть от простой к сложной
- Новые структуры защищаются нишеванием, пока не окрепнут
- Результат — минимально достаточная архитектура, а не заданная заранее

Ключевая идея

Начинать с простого и усложнять по мере необходимости — это регуляризация: сложность появляется только тогда, когда за неё заплачено ростом качества.

- Слишком подробная функция — вы просто запрограммировали решение вручную
- Слишком общая — алгоритм не находит ничего осмысленного
- Алгоритм всегда находит **лазейку**, а не то, что вы имели в виду
- Настройка целевой функции часто занимает больше времени, чем сама оптимизация

Кейс

Робот, награждаемый за пройденное расстояние, обучается вращаться на месте: колёса крутятся, одометрия растёт, задача формально выполнена.

- Задачу решает группа, а награда общая
- Непонятно, чей вклад был полезным, а чей вредным
- При ограниченной связи робот не отличает свои последствия от чужих
- Индивидуальная оценка порождает эгоизм, групповая — иждивенчество

Ключевая идея

Это центральная трудность коллективного обучения. Она встречается и в эволюции роёв, и в многоагентном обучении с подкреплением — под разными именами, но с одной сутью.



При ограниченной связи или частичной наблюдаемости робот не может отличить изменения среды, вызванные его собственными действиями, от вызванных действиями соседей. Поэтому награды и штрафы оказываются несогласованными с реальным вкладом.

Однородное обучение

- Все роботы получают одинаковый контроллер
- Пространство поиска компактно
- Быстрая сходимость
- Специализация невозможна

Разнородное обучение

- У каждого свой набор параметров
- Пространство растёт с числом роботов
- Сходимость медленная
- Возможна специализация и разделение труда

- Одно испытание длится десятки секунд — испытаний нужны тысячи
- Оценка приспособленности зашумлена: тот же контроллер даёт разный результат
- Шум сбивает отбор: везучая посредственность вытесняет хорошее решение
- Лечится повторными оценками, но это ещё дороже

Факт

Из-за шума алгоритм может уверенно выбрать худший контроллер, которому просто повезло в единственном испытании. Шумоустойчивые схемы перепроверяют лидеров дополнительно.



Инженерная ситуация

Контроллер, выращенный в симуляторе, отлично работает там и полностью проваливается на реальном роботе. Наиболее вероятная причина?

А

Слишком мало поколений — надо учить дольше

В

Алгоритм переобучился на особенности симулятора

С

Неправильно выбран метод отбора

Д

Слишком большая популяция



Инженерная ситуация

Контроллер, выращенный в симуляторе, отлично работает там и полностью проваливается на реальном роботе. Наиболее вероятная причина?

A

Слишком мало поколений — надо учить дольше

B

Алгоритм переобучился на особенности симулятора

C

Неправильно выбран метод отбора

D

Слишком большая популяция

ОТВЕТ

Переобучение на симулятор: эволюция нашла и использовала неточности модели, которых в реальности нет. Это и есть разрыв между симуляцией и реальностью.

- Симулятор всегда неточен: трение, шум датчиков, задержки
- Эволюция находит и **эксплуатирует** эти неточности
- Рандомизация параметров среды делает решение робастнее
- Дообучение на реальном роботе замыкает петлю

Ключевая идея

Чем точнее контроллер подогнан под симулятор, тем хуже он переносится. Полезно намеренно портить симуляцию, заставляя решение работать в целом диапазоне условий.

- Контроллер собирается из проверенных поведенческих блоков
- Оптимизируется не всё подряд, а структура и параметры сборки
- Пространство поиска резко сужается
- Переносимость на реальных роботов заметно выше

Ключевая идея

Модульность здесь играет роль регуляризации: ограничивая пространство поиска проверенными блоками, мы не даём алгоритму переобучиться на артефакты симулятора.

12 Сравнение подходов к проектированию поведения

Подход	Кто задаёт способ	Переносимость	Прозрачность
Ручное программирование	Инженер полностью	Высокая	Полная
Модульный автодизайн	Инженер задаёт блоки	Хорошая	Хорошая
Нейроэволюция	Алгоритм полностью	Слабая	Низкая
Обучение с подкреплением	Алгоритм полностью	Средняя	Низкая

Чем больше свободы у алгоритма, тем интереснее найденные решения и тем труднее их перенести и объяснить.

“

Эволюция изобретательна ровно настолько, насколько точна ваша целевая функция. Она всегда найдёт лазейку, если вы её оставите.

Главный практический вывод лекции

- Уместна: нет модели, нет градиента, много локальных оптимумов
- Уместна: нужно исследовать непривычные, неочевидные решения
- Не уместна: есть градиент и работает обычная оптимизация
- Не уместна: требуется гарантия качества или сертификация

Факт

Если задача решается градиентным методом, эволюция почти всегда проиграет ему на порядки по числу вычислений целевой функции.

Выводы по лекции

- 1 Эволюционные методы работают там, где нет модели и нет производной, — ценой большого числа испытаний
- 2 Кодирование решения влияет на успех сильнее, чем выбор операторов
- 3 Отбор создаёт давление к улучшению, мутация обеспечивает разнообразие; их баланс определяет исход
- 4 СМА-ES и дифференциальная эволюция сами настраивают масштаб шага и остаются эталонами для непрерывных задач
- 5 В многокритериальных задачах ответом служит фронт Парето, а не единственное решение
- 6 В эволюционной робототехнике инженер задаёт цель, а не способ; за это приходится платить прозрачностью
- 7 Две главные трудности коллективного обучения — назначение вклада и разрыв между симуляцией и реальностью
- 8 Модульный автоматический дизайн ограничивает пространство поиска и тем самым улучшает переносимость

Вопросы для самопроверки

1. В каких случаях эволюционный алгоритм предпочтительнее градиентного метода?
2. Почему выбор кодирования решения важнее выбора операторов?
3. Зачем нужен элитизм и что происходит без него?
4. В чём смысл правила одной пятой?
5. Почему дифференциальная эволюция почти не требует настройки шага?
6. Что такое фронт Парето и почему на нём нет лучшей точки?
7. Приведите пример лазейки в целевой функции робота
8. В чём состоит проблема назначения вклада в группе роботов?
9. Почему более долгое обучение усугубляет разрыв между симуляцией и реальностью?
10. Как модульный автоматический дизайн повышает переносимость решения?